



FedMini: A Federated Learning Framework with Partial Network Updates and Parameter-Wise Personalization

Zilong Guo¹, Jie Zhang^{2(✉)}, Canran Li¹, Huiyong Liu¹, and Lanlan Rui¹

¹ State Key Laboratory of Networking and Switching Technology, Beijing University of Posts
and Telecommunications, Beijing, China

² Sinomach Digital Technology Co., Ltd., Nanjing 210042, China
zhangjie1@sinomach.com.cn

Abstract. Federated Learning (FL) has gained widespread adoption due to its privacy-preserving capabilities and collaborative learning framework. Nevertheless, in edge computing deployments, significant statistical heterogeneity across client data distributions combined with stringent communication constraints severely undermines the effectiveness of cross-client collaborative model training. To solve this problem, we introduce a unified framework for personalized federated learning that integrates partial network update with fine-grained parameter-level adaptation. We update network layers sequentially across communication rounds, alleviating layer mismatch and adopt a dynamic freezing strategy to enhance convergence. Concurrently, we employ a Fisher information-based sensitivity metric to identify sensitive parameters for each client. These parameters are selectively aggregated among clients with similar data distributions, while the remaining parameters are shared globally. Theoretical analysis reveals that this hybrid approach mitigates client heterogeneity at multiple scales. Our algorithmic contributions emphasize personalization with limited communication cost. Experimental results show that our method has better performance compared to state-of-the-art methods.

Keywords: Personalized Federated Learning · Layer-wise Updates · Parameter-level Collaboration · Fisher Information

1 Introduction

Federated Learning, a privacy-preserving distributed machine learning framework, enables collaborative model training across edge devices without sharing raw data, finding extensive applications in healthcare, finance, smart manufacturing, and other domains.

Nevertheless, the inherent heterogeneity of edge devices in computational capabilities, communication bandwidth, and data distribution poses significant challenges to FL performance. Specifically, non-independent and identically distributed (non-IID) local data deteriorates global model convergence, while constrained communication resources

exacerbate training latency and energy consumption. To address these issues, researchers have proposed various optimization strategies including model compression, gradient quantization, and adaptive aggregation mechanisms, aiming to enhance the efficiency and accuracy of FL in heterogeneous edge environments.

To address the aforementioned challenges, this paper proposes a personalized federated learning framework that synergistically integrates parameter collaboration mechanisms with partial network training strategies. The proposed approach aims to simultaneously enhance communication efficiency and model accuracy in heterogeneous edge environments through two core innovations:

First, the framework implements partial network updates via a layer selection strategy to reduce the communication payload per aggregation round. This selective parameter transmission mechanism effectively minimizes communication overhead while maintaining essential feature representation capabilities. Second, we introduce a parameter sensitivity analysis (PSA) module that quantitatively evaluates the collaboration potential of different model parameters. The system dynamically determines the optimal collaboration scope based on the degree of heterogeneity and the training progress.

2 Related Work

The inherent non-IID characteristics of client data in real-world scenarios often degrade the performance of conventional federated learning aggregation on personalized tasks. This has motivated the development of Personalized Federated Learning (PFL) [1], which seeks to tailor dedicated models for heterogeneous clients. Current PFL methodologies primarily adopt two paradigms.

Partial-layer Aggregation: FedBN [2] preserves client-specific Batch Normalization statistics to mitigate feature distribution shifts across clients. CGPFL [3] introduces “contextualized generalization” through maintaining multiple context vectors, enabling clients to adaptively combine global and local models. FedPart [4] addresses layer mismatch by selectively updating partial layers while anchoring alignment through frozen layers. FedPer [5] freezes global feature extractors while updating local classifiers. Such methods effectively reduce transmission overhead, yet they are still limited by issues such as over-reliance on pre-defined architectures and insensitivity to the continuously evolving training dynamics.

Collaboration Scope Restriction: FedCAC [6] quantifies parameter sensitivity to non-IID data for determining collaboration-critical parameter sets. IFCA [7] clusters clients into groups for multi-center model training. APPLE [8] clients download the core models of other clients from the server and weights these models by learning an directed relational vector. FedACS [9] incorporates an attention-based client selection mechanism that dynamically identifies optimal collaborators by evaluating pairwise model similarity. PFedCM [10] facilitates client clustering by probabilistically estimating client-specific class distributions, and dynamically selects semantically coherent client subsets for participation in each communication round. However, these methods generally neglect the semantic hierarchy inherent in deep neural networks, struggling to balance global knowledge integration with personalized adaptation requirements.

3 Method

3.1 Problem Formulation

Following the paradigm of PFL, we consider a federated system comprising N distributed clients. Each client i possesses a local dataset D_i and maintains a personalized model parameterized by w_i . The joint objective aims to collaboratively learn personalized models. In conventional FL frameworks, global model parameters w_{glob} are typically optimized through weighted aggregation:

$$w_{glob}^{t+1} = \sum_{i=1}^N \frac{|D_i|}{\sum_{j=1}^N |D_j|} w_i^t \quad (1)$$

However, this full-model averaging proves suboptimal under non-IID data distributions due to parameter divergence across clients. While purely local training avoids aggregation errors, it risks overfitting to limited local samples and fails to leverage potential cross-client knowledge transfer. Let $L_i(w_i, D_i)$ denote the loss function of client i ,

the principal objective of our framework is to reduce communication overhead and solve the following problem:

$$\min_{w_1, w_2, \dots, w_N} \sum_{i=1}^N L_i(w_i, D_i) \quad (2)$$

3.2 Overview

In this section, we introduce FedMini, a novel approach that addresses the challenges of communication efficiency and personalization in federated learning. Figure 1 illustrates the workflow of FedMini. Unlike traditional full-network updates, FedMini employs a selective layer aggregation mechanism. Prior to each communication round, the server dynamically notifies participating clients about the specific network layers designated for aggregation. Subsequently, clients perform localized training exclusively on the specified layers while maintaining parameter freezing in other layers, awaiting server-side aggregation. To address the challenges of non-IID data distribution, we propose a novel sensitivity analysis algorithm that systematically categorizes network layer parameters into two distinct groups: non-IID-sensitive parameters and non-IID-robust parameters. The sensitive parameters updates are selectively combined only among clients with statistically similar parameter distributions. The robust parameters need global aggregation to achieve collaboration within the maximum range.

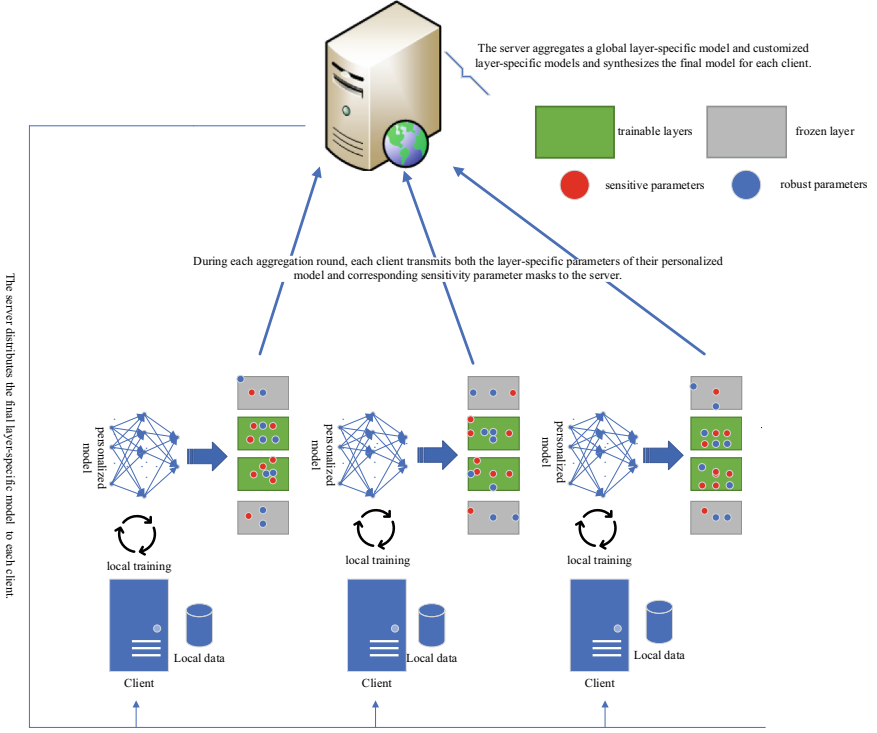


Fig. 1. The system workflow of FedMini

3.3 Trainable Layers Selection

This section proposes a novel partial network update framework based on phased training and dynamic freezing, aiming to significantly reduce communication and computational overhead while maintaining model performance, with enhanced global convergence and stability. We divide the layers of model into trainable layers and frozen layers. Let $w^{(l)}$ denote the parameters of the l -th layer, and D_i represent the local data distribution on client i . The local training objective during communication round r , where layer l is selected for training, is formulated as:

$$\min_{w_i^{(l)}} E_{x \sim D_i} [L(x; w^{(1)}, \dots, w^{(l)}, \dots, w^{(L)})] \quad (3)$$

where the model parameters in layers $k \neq l$ are fixed and excluded from backpropagation. After local training, each client uploads only the updated parameters, and the server aggregates these updates to obtain the global model for layer l :

$$w_{glob}^{(l)} = \frac{1}{N} \sum_{i=1}^N w_i^{(l)} \quad (4)$$

All other layers $w^{(k)}$, $k \neq l$ are kept unchanged in this round.

Sequential Layer-wise Training Phase. Traditional full-network update strategies are prone to the “layer-wise collaboration mismatch” issue, where distinct hierarchical feature representations develop across clients during local training. The server aggregation process overlooks these inter-layer semantic relationships, leading to slow convergence and compromised model performance. To address this, we introduce a sequential layer-wise training mechanism. Our approach is grounded in the empirical observation that neural network convergence typically follows intrinsic sequence, where shallow layers stabilize earlier than deep layers during training. Guided by this intrinsic regularity, our sequential layer-wise network updating mechanism deliberately mimics the complete network’s convergence trajectory through ordered layer activation. While single-round sequential updating often proves insufficient for comprehensive layer-wise training, leading to constrained feature representation capabilities and potential performance degradation (particularly problematic for deeper layers during early training stages due to parameter update insufficiency), we employ a structured cyclic layer-wise update strategy that alternates between full-model training and partial-layer updates. Specifically, we define each cycle to consist of $q + k \bullet m$ communication rounds, where m is the total number of model layers and k is the number of training rounds allocated to each layer in single cycle. The first q rounds of each cycle are used for full-model training to promote initial convergence and facilitate global knowledge sharing. The remaining $k \bullet m$ rounds of the cycle are dedicated to single-layer training in a sequential manner.

Dynamic Freezing Convergence Phase. While the sequential layer-wise training strategy effectively reduces computation and communication costs and facilitates layer-wise feature stabilization, it introduces accuracy fluctuations at cycle transitions and lacks a convergence guarantee for the full model. To mitigate these drawbacks and ensure stable and efficient final convergence, we introduce a dynamic freezing phase that progressively locks the parameters of converged layers during full-network training. This phase transitions from partial to full-network training, allowing all layers to be updated simultaneously while dynamically monitoring the convergence status of each layer individually. Specifically, we adopt a gradient-based convergence indicator defined as the relative decrease in gradient norm over successive communication rounds. Let $\|\nabla L_j^r\|$ denote the average gradient norm of the l -th layer at communication round r . The convergence indicator Δ_j^r is defined as

$$\Delta_j^r = \frac{\|\nabla L_j^{r-1}\| - \|\nabla L_j^r\|}{\|\nabla L_j^{r-1}\| + \epsilon} \quad (5)$$

where ϵ is a small constant set to 0.001 in our experiments for numeric stability. A layer is considered converged and eligible for freezing if $\eta_t^{(l)} \leq \delta$, where τ is a pre-defined threshold. Additionally, a layer is frozen only if all preceding layers have already been frozen, preventing disruptive layer-wise dependencies. Once a layer is frozen, it is excluded from future backpropagation and parameter aggregation. As the training progresses, the number of trainable layers gradually decreases, leading to significant reductions in computational and communication costs.

The detail of the layer selection process is summarized as follows

Algorithm 1: Trainable Layer Selection

Input: Total rounds T , number of layers m , per-round repeat k , phase switch γ , freezing threshold δ , constant ε

Output: Trainable layer set per round

Define cycle length $C = q + k \cdot m$.

for round $t = 1$ to T **do**

if $t < \gamma \cdot C$

if $t \bmod C < 5$ **then**

 Train all layers.

else

$l_t = ((t \bmod C) - q)/k \bmod m$

 Freeze all layers except l_t .

else

foreach unfrozen layer l_j **do**

 Compute gradient norm $\|\nabla L_j^r\|$.

 Compute Δ_j^t by Eq.(5).

if $\Delta_j^t \geq \delta$ and previous layers are frozen **then**

 Freeze l_j .

 Clients train on current trainable layers.

3.4 Parameter Sensitive Evaluation

The goal of parameter sensitive evaluation is to identify which parameters of the model are important and sensitive to non-IID during local epoch. In this stage, each client focuses only on the model's trainable layers. Within these layers, the sensitivity of each parameter is assessed by combining a Fisher information estimate with the magnitude of the parameter's own update during local training. This combined metric serves to highlight parameters that are both inherently influential (according to the Fisher information) and that have undergone substantial change during local epoch.

Sensitivity Scoring. Following established methodologies in previous research [11, 12] on importance estimation, we quantify parameter sensitivity through variation in the loss function under nullification. Let $\theta = \{\theta_1, \theta_2, \dots, \theta_n\}$ denote the whole parameter of the trainable layers and L denote the loss function. Typically, the sensitivity of the i -th parameter can be defined as

$$s_i = |L(\theta) - L(\theta_1, \dots, \theta_{i-1}, 0, \dots, \theta_n)|. \quad (6)$$

However, according to Eq. (6), evaluating the sensitivity of each parameter requires an additional forward propagation, which is unacceptable in edge scenarios with limited computing power. We consider approximating the estimation of parameter sensitivity

from the following two aspects. First, we use the first-order Taylor approximation to estimate s_i as

$$s_i \approx \left| \nabla_{\theta_i} L(\theta) \bullet \theta_i \right|. \quad (7)$$

In our framework, each communication round encompasses E epochs of local training. Thus, when computing the sensitivity of parameter θ_i in the $r - th$ communication round, we can substitute $\nabla_{\theta_i} L(\theta)$ with the variation of parameter θ_i over e epochs of local updates. The sensitivity of parameter θ_i in $r - th$ communication round can then be written as

$$s_i^r = \left| \left(\theta_i^r - \theta_i^{r-1} \right) \bullet \theta_i \right|. \quad (8)$$

Moreover, we introduce the Fisher Information Matrix (FIM) as a quantitative metric for parameter importance. This metric reflects the curvature of the loss surface and quantifies how sensitive the likelihood is to changes in a given parameter. Specifically, the FIM for client i is defined as the expected covariance of the log-likelihood gradients

$$F_i = \frac{1}{n} \sum_{j=1}^n E_{(x,y)} [\nabla_{\tilde{W}_i} \log P(yx_{i,j}, W) \nabla_{\tilde{W}_i} \log P(yx_{i,j}, W)^\top] \quad (9)$$

where $\tilde{W}_i$ denotes the locally optimized parameters of client i and $x_{i,j}$ denotes the $j - th$ local data sample of client i . The second-order interpretation of the FIM lies in its approximation of the Hessian matrix of the loss function, reflecting the sensitivity of model confidence to minor parameter perturbations. A parameter with a high Fisher value is generally more important for preserving the learned local model behavior, indicating that even small perturbations to this parameter can lead to significant performance degradation. However, computing and communicating the full FIM would require $O(d^2)$ bits, which is unacceptable. To solve this problem, we approximate it with the diagonal Fisher, which retains only diagonal elements. In practice, the calculation method of the diagonal fisher is computed by taking the average of the squared gradients over the local data at each client. The diagonal fisher for param θ_k of client i can be expressed as

$$diag(\tilde{F}_i)_{\theta_k} = \frac{1}{n} \sum_{j=1}^n (\nabla_{\theta_k} L(f(W, x_{ij}), y_{ij}))^2. \quad (10)$$

To unify these two perspectives, we need a combined sensitivity score. Since fisher information and param update magnitudes may vary in scale, both are independently normalized to the range $[0,1]$ using min-max normalization. Specifically, for each parameter k , the normalized scores in the r -th communication round are computed as

$$\tilde{F}_k^r = \frac{F_k^r - \min_j F_j^r}{\max_j F_j^r - \min_j F_j^r + \varepsilon}, \tilde{s}_k^r = \frac{s_k^r - \min_j s_j^r}{\max_j s_j^r - \min_j s_j^r + \varepsilon}, \quad (11)$$

where the minimum and maximum are computed over all trainable parameters on the client in current round, and ε is a small constant to avoid division by zero. A combined sensitivity score is then calculated for each parameter as the weighted average of the two normalized components

$$Sc_k = \alpha \cdot \tilde{F}_k^r + (1 - \alpha) \cdot \tilde{s}_k^r, \quad (12)$$

where $\alpha \in [0,1]$ controls the balance between the two perspectives. After experimental verification, the weights are set to 0.25.

3.5 Aggregation Strategy

From an optimization perspective, parameters critical for localized task performance may experience accuracy deterioration when subjected to aggregation from clients under non-IID data distributions.

Dynamic Dual-Level Aggregation Framework. Based on the computed scores Sc_k , the client selects the top β fraction of parameters as sensitive. This is achieved by sorting all parameters by descending sensitivity and marking the top $\lfloor \beta \cdot M \rfloor$ parameters, where M is the number of trainable parameters in the selected layers. A binary mask is generated to label each parameter as either sensitive (1) or in sensitive (0). Rather than using a fixed parameter selection ratio, we implement an adaptive mechanism that adjusts the value of β based on the training progress

$$\beta_r = \begin{cases} \beta_{base} \cdot 0.8, & \text{if } r < 0.3 \cdot T \\ \beta_{base}, & \text{if } 0.3 \cdot T \leq r < 0.7 \cdot T \\ \min(\beta_{base} \cdot 1.2, 0.8) & \text{if } r \geq 0.7 \cdot T \end{cases}, \quad (13)$$

where β_{base} is a base selection ratio, r is the current round, and T is the total number of the communication rounds. This approach enables more effective knowledge transfer in the early stages and better personalization in later stages.

In our framework, sensitive parameters are restricted to aggregation exclusively among clients exhibiting data distribution congruence, while insensitive parameters undergo conventional global synchronization via federated averaging. Experimental results demonstrate that clients with similar data distributions often exhibit analogous positional distributions of sensitive parameters. To regulate the scope of sensitive parameter collaboration among clients, we propose a time-adaptive aggregation scheme that progressively tightens collaboration boundaries as training proceeds. In each communication round, clients evaluate their sensitive parameters and send binary masks indicating

which parameters are marked as sensitive. The server then computes pairwise overlap scores among all clients to assess their parameter-level similarity. The overlap score between client i and client j ($i \neq j$) can be denote as

$$O_{i,j}^r = \frac{\sum_{k=1}^d I(M_{i,k}^r = 1 \wedge M_{j,k}^r = 1)}{\sum_{k=1}^d I(M_{i,k}^r = 1 \vee M_{j,k}^r = 1)}, \quad (14)$$

where $M_{i,k}^r$ denotes the k -th param in the binary mask of client i . Let O_{avg}^r represent the mean of these scores. Instead of relying on the absolute maximum overlap score, which can be sensitive to outliers, we adopt a more robust approach by using the 90th percentile $Q_{95}(O^r)$ as the upper anchor. To gradually restrict collaboration to more similar peers, the collaboration threshold is defined as

$$Threshold^r = O_{avg}^r + \left(Q_{90}(O^r) - O_{avg}^r \right) \cdot \left(\frac{r}{\mu \bullet T} \right), \quad (15)$$

where T is the total number of training rounds and μ is a hyperparameter controlling the aggressiveness of threshold growth. Our design base on the intuition that in the early training, broad collaboration promotes generalization and accelerates convergence, while later rounds benefit more from personalized refinement.

Once the sensitive parameter masks have been received from all clients, the server can establish the collaborative client-set for each client. The server then performs the aggregation of both sensitive and insensitive parameters. For each client i , the personalized sensitive parameters are computed by averaging over the client's collaborator set, while the insensitive parameters are computed by global averaging across all clients.

In the r -th communication round, the server synthesizes a single final model w_i^{r+1} for each client by directly applying the client's binary mask M_i^r as

$$w_i^r[k] = \begin{cases} w_{global}^{r-1}[k], & \text{if } M_i^{r-1}[k] = 0, \\ w_{personal}^{r-1}[i][k], & \text{if } M_i^{r-1}[k] = 1, \end{cases} \quad \forall k = 1, \dots, d. \quad (16)$$

The detail of the aggregation process is summarized as follows

Algorithm 2: Aggregation on trainable Layers

Input: Client data D_i , round t , total rounds T , base ratio β_{base} , weight α , hyperparameter μ

Output: Final personalized model w_i^{t+1} for each client i

Train on selected layer set L_t .

foreach client i in parallel **do**

foreach parameter $k \in L_t$ **do**

 Compute the combined sensitivity score Sc_k by Eq. (11)

 Compute the collaboration threshold by Eq. (14)

 Compute β by the Eq. (12)

 Select top- β of Sc_k as sensitive;

 Create binary mask M_i^t and upload it with θ_i^t to server

 Server computes pairwise overlap score $O_{i,j}^t$ by Eq. (13)

 Compute the collaboration threshold by Eq. (14)

foreach client i **do**

 Build the collaborative client-set C_i^t

foreach parameter $k \in L_t$ **do**

if $w_i^t[k] = 1$ **then**

 aggregate with C_i^t

else

 aggregate with all clients

 Construct and distribute final model w_i^{t+1} to client i

4 Experiments

To validate the effectiveness of our method, we conduct experiments on image classification under heterogeneous federated learning settings. We evaluate our approach on four commonly used datasets, including CIFAR-10, CIFAR-100, and Tiny ImageNet. We used the ResNet network for experiments, specially ResNet18 for Tiny ImageNet and ResNet-8 for CIFAR-10 and CIFAR-100. we utilize the Adam optimizer with a learning rate of 0.01. In the experimental setup, we choose 50 clients, with local epochs to be 6. Each client has 500 training samples and 100 test samples. Test data and training data have the same data distribution. We set the total communication round to 300 to ensure full convergence. We evaluate the averaging test accuracy across all clients in each communication round and select the best accuracy as the final. We compare our method with five representative approaches, namely FedAvg [13], FedPer [5], FedBN [2], FedACS[9], FedCAC [6]. we adopt two non-IID scenarios for experiments, including pathological non-IID and Dirichlet non-IID. Hyperparameters setting of our experiments are shown in Table 1. Each experiment is conducted three times with different random seeds, and the mean accuracy and standard deviation are reported.

Table 1. Hyperparameters settings

Hyperparameter	Value
q	5
k	3
γ	6
δ^r	$\mathcal{Q}_{50}(\Delta_j^r)$
β_{base}	0.5
μ	0.6

In the pathological non-IID scenario, we sampled data with label amount 2/10/20 for each client on CIFAR-10/ CIFAR -100/Tiny ImageNet. The data distributions among clients are very different. As can be seen from the results in Table 2, Fedmini outperforms other baseline methods. FedBN and FedPer enhance performance by locally training layers that are sensitive to non - IID data. FedACS reduces the influence of heterogeneous data by choosing clients with similar data distributions for collaboration. FedCAC differentiates the training process by classifying parameters into sensitive and insensitive ones, thereby controlling the scope of client collaboration. Only our method takes into account the layer mismatch issue and controls the aggregation scope at the parameter level simultaneously, ultimately achieving optimal performance across all datasets.

Table 2. The test accuracy (%) in under pathological non-IID.

	CIFAR10	CIFAR100	Tiny ImageNet
FedAvg	52.78 $\pm$ 2.92	25.87 $\pm$ 0.97	15.23 $\pm$ 0.48
FedPer	87.23 $\pm$ 0.97	63.33 $\pm$ 0.26	39.47 $\pm$ 0.51
FedBN	87.12 $\pm$ 0.71	63.09 $\pm$ 0.14	40.61 $\pm$ 0.47
FedACS	88.58 $\pm$ 0.91	65.76 $\pm$ 0.31	39.97 $\pm$ 0.42
FedCAC	89.37 $\pm$ 1.03	67.37 $\pm$ 0.19	40.98 $\pm$ 0.27
FedMini	89.97 $\pm$ 1.26	68.79 $\pm$ 0.17	42.03 $\pm$ 0.19

In the Dirichlet non-IID scenario, We conduct experiments with α values of {0.1, 0.5, 1.0}. As can be seen from Table 3, FedMini obtained the best performance under all settings in our experiments. FedMini performs better when the degree of data heterogeneity is higher, and its advantages over other baseline methods are more obvious in complex datasets.

Table 3. The test accuracy (%) in under Dirichlet non-IID.

α	CIFAR10			CIFAR100			Tiny ImageNet		
	0.1	0.3	0.5	0.1	0.3	0.5	0.1	0.3	0.5
FedAvg	54.31 $\pm$ 2.73	57.48 $\pm$ 1.36	60.4 $\pm$ 1.36	34.16 $\pm$ 0.70	33.13 $\pm$ 0.53	32.78 $\pm$ 0.43	21.26 $\pm$ 1.28	20.75 $\pm$ 1.17	20.32 $\pm$ 0.91
FedPer	83.23 $\pm$ 0.57	70.51 $\pm$ 0.49	67.31 $\pm$ 0.39	51.29 $\pm$ 0.88	37.83 $\pm$ 0.54	27.31 $\pm$ 0.93	30.97 $\pm$ 0.43	18.19 $\pm$ 0.47	11.69 $\pm$ 0.39
FedBN	83.93 $\pm$ 1.59	72.41 $\pm$ 0.78	66.76 $\pm$ 1.03	52.13 $\pm$ 0.69	41.93 $\pm$ 0.34	35.41 $\pm$ 1.03	32.83 $\pm$ 0.81	23.17 $\pm$ 0.53	18.31 $\pm$ 0.47
FedACS	84.73 $\pm$ 1.29	71.93 $\pm$ 0.81	68.07 $\pm$ 0.67	54.31 $\pm$ 0.71	40.31 $\pm$ 0.54	28.51 $\pm$ 0.39	29.25 $\pm$ 0.33	19.33 $\pm$ 0.81	14.23 $\pm$ 0.38
FedCAC	85.99 $\pm$ 1.18	73.37 $\pm$ 0.93	69.23 $\pm$ 0.44	56.57 $\pm$ 1.39	42.53 $\pm$ 0.71	37.13 $\pm$ 0.73	38.79 $\pm$ 1.07	28.94 $\pm$ 0.71	23.04 $\pm$ 0.41
FedMini	86.93 $\pm$ 0.91	75.43 $\pm$ 0.68	70.31 $\pm$ 0.49	58.23 $\pm$ 1.07	45.03 $\pm$ 0.35	38.21 $\pm$ 0.64	40.29 $\pm$ 0.93	30.57 $\pm$ 0.67	24.31 $\pm$ 0.37

We quantitatively evaluated the communication overhead of FedMini against full-network update baselines (exemplified by FedAvg). As can be seen from Table 4, FedMini provides about 60% reduction in communication costs for ResNet-8 and 73% reduction for ResNet-18.

Table 4. The Comparison of communication cost

	CIFAR10	CIFAR100	Tiny ImageNet
FedAvg	46.7	47.3	528.5
FedMini	18.4	18.8	142.7

In FedMini, β_{base} is an important hyperparameter, which controls the proportion of sensitive parameters. We conduct the experiment in Dirichlet non-IID scenario with $\alpha = 0.1$ to evaluate the effect of it. As can be seen from Table 3, the accuracy is highest when β_{base} equals around 0.5. This is because when β_{base} is set too small, some sensitive parameters will be globally aggregated, damaging the model performance. When β_{base} is set too large, those insensitive parameters lack sufficient collaboration (Fig. 2).

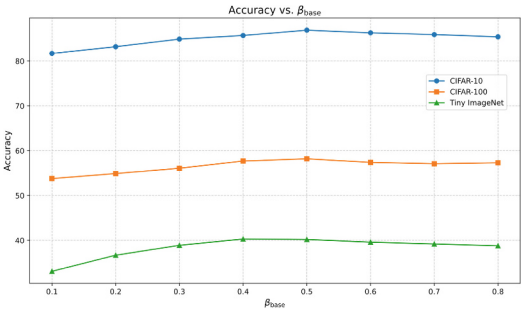


Fig. 2. The effect of β_{base}

5 Conclusion

We have presented a novel personalized federated learning framework that unifies partial network updates and adaptive parameter-level collaboration. By dynamically freezing network layers and applying a Fisher information-based metric to identify sensitive

parameters, our approach addresses client heterogeneity at multiple scales and balances global sharing with client-specific adaptation. These contributions enhance personalization communication efficiency. Future work will explore theoretical convergence analysis, adaptive hyperparameter tuning, and broader evaluations across diverse federated learning scenarios.

Acknowledgment. This work was supported by the National Natural Science Foundation of China (62471051). Jie Zhang is the corresponding author with email: zhangjiei@sinom-ach.com.cn.

References

1. Tan, Y., et al.: Fedproto: federated prototype learning across heterogeneous clients. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 36, pp. 8432–8440 (2022)
2. Li, X., Jiang, M., Zhang, X., Kamp, M., Dou, Q.: FedBN: federated learning on non-iid features via local batch normalization. In International Conference on Learning Representations (202)
3. Tang, X., Guo, S., Guo, J.: Personalized federated learning with contextualized generalization. arXiv preprint [arXiv:2106.13044](https://arxiv.org/abs/2106.13044) (2021)
4. Wang, H., Liu, X., Niu, J.: Why go full? Elevating federated learning through partial network updates. Adv. Neural. Inf. Process. Syst. **37**, 34839–34855 (2024)
5. Arivazhagan, M., Aggarwal, V., Singh, A.K.: Federated learning with personalization layers. arXiv preprint [arXiv:1912.00818](https://arxiv.org/abs/1912.00818) (2019)
6. Wu, X., Liu, X., Niu, J.: Bold but cautious: unlocking the potential of personalized federated learning through cautiously aggressive collaboration. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 19375–19384 (2023)
7. Ghosh, A., Chung, J., Yin, D.: An efficient framework for clustered federated learning. Adv. Neural. Inf. Process. Syst. **33**, 19586–19597 (2020)
8. Luo, J., Wu, S.: Adapt to adaptation: learning personalization for cross-silo federated learning. In: IJCAI: Proceedings of the Conference 2022, p. 2166 (2022)
9. Wang, Z., Zhu, Y., Wang, D.: FedACS: federated skewness analytics in heterogeneous decentralized data environments. In: 2021 IEEE/ACM 29th International Symposium on Quality of Service (IWQOS), pp. 1–10. IEEE (2021)
10. Huang, L., Gou, S., Cao, S.: Personalized federated learning using client clustering for medical image classification. In: Proceedings of the 2024 9th International Conference on Biomedical Imaging, Signal Processing, pp. 46–52.3 (2024)
11. Huang, Y., Chu, L., Zhou, Z.: Personalized cross-silo federated learning on non-iid data. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 35, no. 9, pp. 7865–7873 (2021)
12. Kim, J., Kim, G., Han, B.: Multi-level branched regularization for federated learning. In: International Conference on Machine Learning, pp. 11058–11073. PMLR (2022)
13. McMahan, B., Moore, E., Ramage, D.: Communication-efficient learning of deep networks from decentralized data. Artif. Intell. Stat. 1273–1282. PMLR (2017)